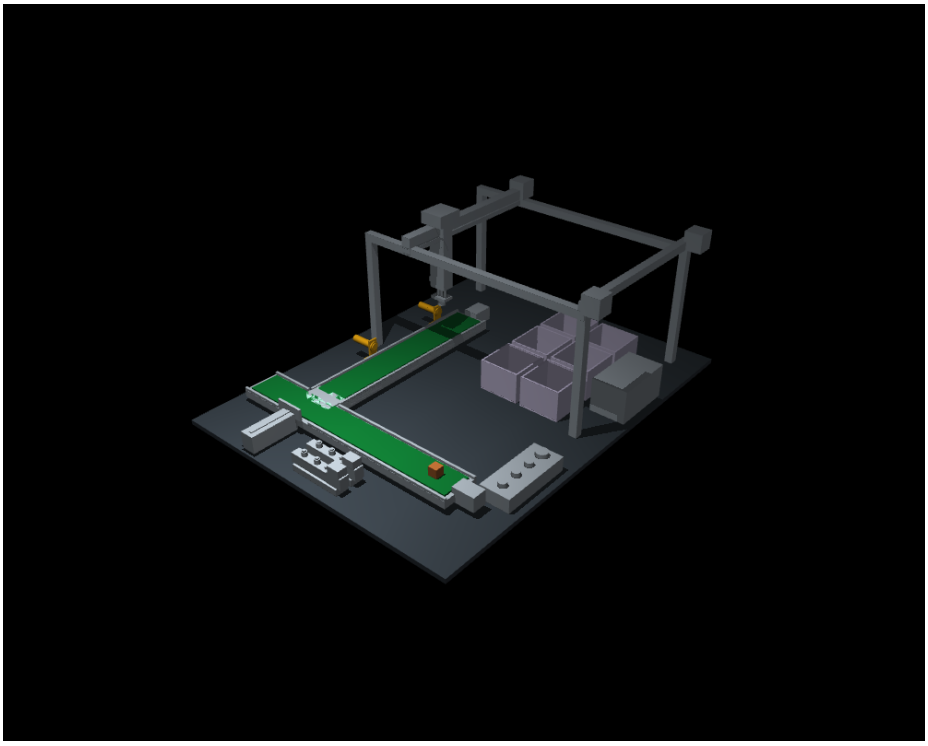
XÂY DỰNG LÕI DIGITAL TWIN

Network-Aware Predictive Digital Twin for Resilient and Self-Reconfigurable Cyber-Physical Production Systems

Hướng dẫn thực hành và nền tảng lý thuyết cho trạm hai băng tải, A1 đẩy phôi, robot XY, A2 nâng hạ và giác hút. Phiên bản 1.1 - cập nhật ngày 24/10/2026.



Việc cần làm ngay

Tạo giao diện tín hiệu, mô hình cảm biến và bộ ước lượng trạng thái. Bộ điều khiển tiếp theo chỉ được dùng các tín hiệu có thể đo trên máy thực. Vị trí phôi chính xác trong MuJoCo chỉ dành cho bộ đánh giá kết quả.

Sau đó: máy trạng thái điều khiển → ghi và phát lại dữ liệu → đồng bộ có xét độ trễ → phát hiện bất thường → dự báo nhiều kịch bản → giám sát và tự tái cấu hình phần mềm.

Tài liệu phân biệt ba loại số liệu: cấu hình đã đọc từ mã nguồn; kết quả chạy mô phỏng; và mục tiêu nghiệm thu đề xuất. Chưa có số đo hệ thật để chứng minh độ chính xác hoặc mức tăng năng suất.

Dành cho người mới: đọc phần A1-A9 trước và làm từng bài. Sau đó dùng phần 01-14 để tra kiến trúc, công thức, chỉ số và nguồn tham khảo. A1-A9 là hướng dẫn triển khai; các chức năng tương lai được ghi rõ, không phải tính năng đã có.

A1 Bắt đầu từ đâu khi chưa biết lập trình?

Mục tiêu buổi đầu: tự chạy mô hình, lưu được dữ liệu và hiểu chương trình quyết định gắp phôi bằng cách nào. Chỉ chuyển bài khi đã đạt điều kiện hoàn thành. Các phần ước lượng, phát hiện lỗi và dự báo cần viết thêm Python; hiện mới có mô hình vật lý và chu trình thử một phôi.

Tên	Hiểu đơn giản	Bạn dùng để làm gì?
MuJoCo	Máy sản xuất ảo.	Quan sát băng tải, xi lanh, robot và phôi.
Python	Chương trình ra lệnh cho máy ảo.	Đọc, sửa và bổ sung thuật toán.
WSL	Môi trường Linux trên Windows.	Chạy Python và MuJoCo đã cài.
PowerShell	Cửa sổ nhập lệnh trên Windows.	Dán các lệnh thực hành trong phần A.
CSV	Bảng dữ liệu theo thời gian.	Mở bằng Excel để đối chiếu hoạt động.
Digital Twin	Mô hình nhận dữ liệu để ước lượng và dự báo.	Phần lõi sẽ xây dựng sau chu trình hiện tại.

Mở đúng thư mục

Nhấn Windows + E để mở File Explorer. Bấm thanh địa chỉ, dán đường dẫn dưới đây và nhấn Enter. Các tệp .py là mã Python, .xml là mô hình, .csv là dữ liệu; bật hiển thị phần mở rộng tệp để tránh nhầm .py thành .txt.

```
C:\Users\Admin\Documents\ChatGPT\plc\mujoco
```

Trước mắt chỉ quan tâm run_cell.py, production_cell.xml và commissioning.csv. Xem mã bằng trình soạn thảo văn bản; không cần sửa XML trong các bài đầu. Trước mỗi lần sửa, lưu một bản sao mã đang chạy được ở thư mục riêng hoặc tạo phiên bản bằng Git nếu đã biết dùng.

Cần học đến mức nào?

Học lần lượt: biến lưu dữ liệu; True/False; if kiểm tra điều kiện; hàm def; vòng lặp; bảng dictionary; đọc CSV. Chưa cần học hết Python mới bắt đầu. Đến A4-A8, mỗi bài cần thêm mã và kiểm tra; mô tả dữ liệu trong tài liệu là ví dụ thiết kế, không phải lệnh để dán vào PowerShell.

Hoàn thành A1: tìm được thư mục và ba tệp chính, phân biệt cửa sổ PowerShell với cửa sổ xem MuJoCo, hiểu phần nào đã có và phần nào cần lập trình.

A2 Tự chạy một chu trình và lưu kết quả

Bước 1. Mở PowerShell

Nhấn phím Windows, gõ PowerShell và mở ứng dụng. Toàn bộ lệnh trong bài A2 được dán vào PowerShell, không cần mở Ubuntu riêng. Mỗi dòng bên dưới là một lệnh; nhấn Enter sau mỗi dòng.

```
cd C:\Users\Admin\Documents\ChatGPT\plc
$py = "/home/admin/dt-cpps-venv/bin/python"
$app = "/mnt/c/Users/Admin/Documents/ChatGPT/plc/mujoco/run_cell.py"
$log = "/mnt/c/Users/Admin/Documents/ChatGPT/plc/mujoco/bai1_hop1.csv"
wsl -d Ubuntu-24.04 -- $py -u $app --commission --bin 1 --view --log $log
```

$py, $app và $log chỉ là ba biến lưu đường dẫn để lệnh cuối dễ đọc. Trong cùng cửa sổ PowerShell, chúng còn hiệu lực; nếu mở cửa sổ mới, nhập lại các dòng khai báo. Môi trường Python đã được cài trong WSL trên máy này.

Bước 2. Quan sát đúng thứ tự

Băng tải đầu đưa phôi đến vùng đẩy → A1 đẩy sang băng tải còn lại → phôi tới vùng cảm biến → đầu hút hạ và gắp → robot đưa tới hộp 1 → nhả phôi và về gốc. Cửa sổ tự chạy, không cần bấm Run. Khi xong, PowerShell phải có dòng bắt đầu bằng PASS:.

Bước 3. Đóng và chạy lại

Cửa sổ giữ kết quả cuối để xem. Đóng cửa sổ MuJoCo trước khi chạy lại. Nút Run bị vô hiệu hóa trong passive viewer vì Python điều khiển chu trình. Sau khi đóng cửa sổ, có thể chạy lại lệnh cuối; đổi --bin 1 thành --bin 2 để chọn hộp 2 và đổi tên tệp log nếu muốn giữ cả hai kết quả.

Khi gặp lỗi

Nếu báo không tìm thấy tệp, kiểm tra chính tả đường dẫn và tên thư mục. Nếu chưa thấy cửa sổ nhưng lệnh vẫn chạy, chờ đầu ra; không mở liên tục nhiều bản. Nếu chỉ lỗi đồ họa, thử bỏ --view để kiểm tra chu trình không giao diện. Lưu nguyên thông báo lỗi thay vì sửa ngẫu nhiên thông số vật lý.

Hoàn thành A2: phôi được thả đúng hộp, có PASS và thấy bai1_hop1.csv trong thư mục mujoco. CSV ghi sau khi chu trình chạy xong; nếu đang mở bằng Excel và chạy ghi đè bị từ chối, đóng tệp Excel hoặc chọn tên log mới.

A3 Đọc dữ liệu bằng Excel

Bước 1. Mở bảng

Mở bai1_hop1.csv bằng Excel. Nếu dữ liệu dồn vào một cột, dùng Data → From Text/CSV và chọn dấu phân cách là dấu phẩy. Đây là dữ liệu quan sát; chưa cần chỉnh các giá trị trong bảng.

Cột	Ý nghĩa
phase	Công đoạn đang thực hiện.
time_s	Thời gian mô phỏng tính bằng giây.
ps1, ps2	True: có phát hiện vật; False: không phát hiện trong mô hình cảm biến hiện tại.
x_m, y_m	Vị trí hai trục, đơn vị mét.
a1_m	Hành trình xi lanh đẩy, đơn vị mét.
a2_m	Hành trình xi lanh nâng/hạ, đơn vị mét.
vacuum_command	Lệnh hút; True là đã ra lệnh bật, chưa phải xác nhận gắp được.
product_xyz_m	Vị trí thật của phôi trong mô phỏng, dùng làm đáp án để đánh giá.

Bước 2. Tìm ba thời điểm

Tìm dòng đầu tiên ps2 chuyển sang True; dòng đầu tiên vacuum_command chuyển sang True; và dòng sau đó vacuum_command trở lại False. Ghi time_s cùng phase của từng dòng ra một bảng riêng. So sánh với trình tự đã nhìn thấy trong cửa sổ MuJoCo.

Bước 3. Đọc đơn vị

a2_m = 0.060 nghĩa là hành trình khoảng 60 mm theo chiều khớp đã định nghĩa. time_s = 7.0 nghĩa là 7 giây mô phỏng, không phải đồng hồ Windows. Dữ liệu CSV hiện lấy xấp xỉ mỗi 0,1 giây nên không thể từ bảng này khẳng định thời điểm sự kiện chính xác đến 1 ms.

Hoàn thành A3: giải thích được một dòng dữ liệu đang nói robot làm gì; xác định được thời điểm bắt đầu hút và nhả. Phải phân biệt lệnh hút với cảm biến áp suất hoặc bằng chứng phôi đã theo đầu hút.

A4 Tạo lớp tín hiệu giống máy thật

Công việc tiếp theo cần lập trình. Chương trình hiện đọc product_xyz_m để biết chính xác vị trí phôi. Máy thật không tự có giá trị này. Vì vậy cần đưa một lớp tín hiệu vào giữa MuJoCo và bộ điều khiển.

Nhóm dữ liệu	Ai được đọc?	Ví dụ
Tín hiệu có thể đo	Bộ điều khiển, estimator và supervisor.	PS1/PS2, cảm biến hai đầu xi lanh, home đã được xác nhận.
Đáp án mô phỏng	Bộ đánh giá kết quả.	Vị trí thật của phôi, nhãn lỗi và thời điểm chèn lỗi.

Bước 1. Lập bảng tín hiệu

Ghi tên, đơn vị, nguồn và tình trạng có/không có ngoài thực tế. Xi lanh có cảm biến hai đầu và hai cảm biến quang là thông tin người dùng đã xác nhận. Vị trí trục liên tục, encoder vận tốc băng và Vacuum_OK chưa được xác nhận đầy đủ; ghi unavailable khi không có nguồn.

Bước 2. Viết bộ tạo tín hiệu

Tạo sensor_model.py để đọc plant và sinh các tín hiệu giống cảm biến: A1 thu/ra, A2 trên/dưới, PS1/PS2. Vị trí khớp trong MuJoCo có thể dùng bên trong mô hình cảm biến để tạo công tắc nhị phân. Chọn ngưỡng đầu-cuối cùng dung sai và ghi rõ giả định. Không đưa vị trí thật phôi qua lớp này như một phép đo sẵn có.

```
Ví dụ dữ liệu mong muốn, không phải lệnh chạy:
Thời gian: 5,82 s
PS2: Có phát hiện vật
A1 đã thu: Có
A2 ở trên: Có
Chất lượng dữ liệu: good
```

Bước 3. Gắn timestamp và chất lượng

Tạo signals.py để thống nhất tên trường: nguồn, số thứ tự, thời gian lấy mẫu, thời gian nhận, giá trị và quality. Ghi cạnh đổi trạng thái bên cạnh snapshot. Dữ liệu thiếu hoặc quá cũ phải có nhãn riêng; không biến chúng thành False.

Hoàn thành A4: có danh sách đầu vào bộ điều khiển, nguồn của từng tín hiệu và log chứa dữ liệu cảm biến. Các tệp nêu ở bài này là sản phẩm cần tạo, chưa có trong bản lõi hiện tại.

A5 Chia điều khiển thành từng trạng thái

Máy trạng thái có thể hiểu như người vận hành làm theo danh sách: chờ phôi → đẩy → theo dõi → canh gắp → hạ và hút → nâng và di chuyển → nhả → trở về. Chương trình chỉ được chuyển bước khi điều kiện của bước hiện tại đã đạt.

Bước 1. Viết bảng trước khi sửa mã

Câu hỏi	Ví dụ ở trạng thái PUSH
Ra lệnh gì?	Tác động van A1 theo chiều đẩy.
Chuyển bước lúc nào?	Cảm biến A1 hết hành trình ra đã tác động.
Được chờ bao lâu?	Một timeout có ghi nguồn hoặc giả định ban đầu.
Quá thời gian thì sao?	Chuyển HOLD, ghi lý do và thực hiện hành vi giữ đã định nghĩa.

Bước 2. Tách chu trình dài

Tạo sequence.py, bắt đầu bằng WAIT, PUSH, TRACK, PICK, PLACE, RETURN và HOLD. Mỗi lần tick, đọc tín hiệu mới, cập nhật bộ đếm thời gian và quyết định giữ hay chuyển trạng thái. Mỗi trạng thái có hàm hoặc nhánh xử lý riêng; không dùng một lệnh chờ dài làm toàn bộ chương trình mất khả năng kiểm tra lỗi.

Bước 3. Thử timeout

Cho tín hiệu A1 ra hết không xuất hiện ở đầu vào controller trong một lượt thử. Chương trình phải chuyển HOLD với lý do rõ, không tiếp tục coi như A1 đã ra. Chọn timeout sau khi có dữ liệu chuyển động bình thường; chưa có số đo thì phải ghi là tham số thử.

Điểm cần giải quyết ở vùng đẩy

Hiện chưa xác nhận cảm biến riêng báo phôi tới mặt đẩy A1. Bài thử đầu có thể dùng một phôi đặt ban đầu tại vị trí đã biết, cộng ước lượng thời gian vận chuyển. Không được coi thử nghiệm này là đã giải quyết việc cấp phôi ngẫu nhiên liên tục.

Hoàn thành A5: chu trình bình thường vẫn thành công, có thể theo dõi tên trạng thái; thiếu phản hồi sẽ vào HOLD. Lệnh Reset hoặc phục hồi phải có điều kiện rõ, không tự chạy tiếp giữa một tình huống chưa được xác nhận.

A6 Ước lượng vị trí thay vì đọc đáp án

Bước 1. Hiểu bằng một phép tính

Giả sử cảm biến vừa xác nhận phôi đi qua một mốc đã biết; vận tốc ước lượng 0,16 m/s; đã trôi qua 0,5 giây. Quãng đường dự kiến đi thêm là 0,16 × 0,5 = 0,08 m = 80 mm. Đây là cơ sở cho bộ ước lượng ban đầu, chưa cần học máy.

```
s_hat = s_moc + v_hat * (t_hien_tai - t_qua_moc)
sai_so = s_hat - s_that
```

Bước 2. Lưu đủ thông tin

Tạo estimator.py, lưu mốc quan sát gần nhất, thời điểm đi qua, vận tốc ước lượng, vị trí dự đoán và khoảng sai số. Khi ghép được đúng hai cạnh PS1/PS2 cho cùng một phôi, dùng khoảng cách chia thời gian để ước lượng vận tốc trung bình. Nếu thay tốc độ giữa hai mốc thì giả thiết vận tốc hằng không còn đúng.

Bước 3. Thay đầu vào cho logic gắp

Cho sequence dùng s_hat và độ tin cậy để tính vùng gắp. Không đọc product_xyz_m trực tiếp trong quyết định vận hành. Giữ vị trí thật trong logger/evaluator để chấm sai số. Tương tự, quyết định cấp phôi tới A1 cần tuân theo giả thiết quan sát đã ghi ở A5.

Bước 4. Đo kết quả

Chạy các vận tốc thử và mức trượt khác nhau. Ghi sai số theo thời gian, sai số lớn nhất hoặc p95 và số lượt gắp thành công. Nếu không có quan sát mới, khoảng sai số phải tăng. Khi vượt cửa sổ cho phép, chuyển HOLD thay vì dùng một vị trí ước lượng có vẻ chính xác.

Đầu ra cần có	Cách kiểm tra
Vị trí ước lượng	So với vị trí thật do evaluator giữ riêng.
Mức tin cậy	Giảm khi dữ liệu cũ hoặc vận tốc không chắc chắn.
Khả năng điều khiển	Canh gắp bằng ước lượng; công bố điều kiện thành công/thất bại.

Hoàn thành A6: không còn dùng đáp án vị trí phôi để canh gắp; đã có bảng sai số và giới hạn quan sát. Việc chạy thành công ở một tốc độ chưa chứng minh thuật toán đúng ở mọi tốc độ.

A7 Thử lỗi từng loại và ghi phản ứng

Chỉ bắt đầu sau khi chu trình dựa trên tín hiệu đã hoạt động. Mỗi lượt chỉ thêm một lỗi, giữ các điều kiện khác giống lượt bình thường. Lỗi được chèn vào dữ liệu hoặc model theo đúng loại đang thử; không xóa đáp án của evaluator.

Lượt	Lỗi thử nghiệm	Điều cần quan sát
1	PS2 luôn False.	Thiếu sự kiện dự kiến có được phát hiện không?
2	PS2 luôn True.	Tín hiệu kéo dài bất thường có được phát hiện không?
3	Vận tốc thực của băng thay đổi.	Ước lượng cập nhật hay báo sai lệch?
4	Gói dữ liệu đến muộn.	Có giữ timestamp nguồn và biết dữ liệu cũ không?
5	Ngừng gửi dữ liệu.	Heartbeat/timeout có đưa hệ về hành vi đã định nghĩa không?

Bước 1. Làm phát hiện bằng luật

Bắt đầu bằng: đã dự kiến phôi tới nhưng quá cửa sổ vẫn không có PS2; PS2 che tia quá lâu; hoặc không có dữ liệu mới sau hạn cho phép. Threshold và timeout được chọn từ lượt normal và dữ liệu thử, không phải giá trị chính xác có sẵn cho mọi hệ thống.

Bước 2. Ghi bảng cho mỗi lượt

```
Mã lượt | loại lỗi | lúc chèn lỗi | lúc có thể quan sát
Lúc báo lỗi | phản ứng | kết quả phôi | nhận xét
```

Phân biệt thời gian kể từ lúc chèn lỗi và thời gian kể từ lúc lỗi bắt đầu có dấu hiệu. Một PS bị kẹt OFF từ trước chỉ có thể bị nghi ngờ khi dự kiến phải có vật đi qua. Không yêu cầu phát hiện ngay lúc chưa có thông tin phân biệt.

Bước 3. Thử cả trường hợp không có lỗi

Giữ nhiều lượt normal để đếm báo động giả. PS2 không tác động có thể do phôi chưa tới, phôi bị trượt hoặc dữ liệu mất; không vội kết luận đúng cảm biến đã hỏng. Ghi unknown/suspect nếu chưa đủ bằng chứng.

Hoàn thành A7: có kết quả cho từng loại lỗi, thời gian phát hiện và số báo sai. Các chế độ lỗi và luật này cần lập trình thêm; bản run_cell.py hiện chưa có tùy chọn --fault hoặc --delay.

A8 Chạy trước phương án và chọn hành vi

Sau khi ước lượng và phát hiện lỗi hoạt động, tạo các nhánh dự báo từ cùng một thời điểm. Hãy hiểu như việc thử trước ba cách xử lý trên ba bản sao máy ảo, rồi mới quyết định cho plant đang chạy.

Phương án	Thử điều gì?
A: Tiếp tục	Hoàn thành chu trình với chính sách hiện tại.
B: Ngừng cấp mới	Xử lý phôi hiện có mà không đưa thêm phôi vào.
C: HOLD	Giữ theo chính sách đã định nghĩa, chờ dữ liệu hoặc xử lý lỗi.

Bước 1. Tạo bản chụp trạng thái

Checkpoint phải chứa trạng thái vật lý cùng bộ điều khiển, estimator, timer, hàng đợi mạng và tham số thay đổi. Chỉ sao chép vị trí robot là chưa đủ để chạy lại cùng tương lai. Tham khảo phần 09 cho chi tiết MuJoCo.

Bước 2. Viết predictor và bộ đánh giá

Tạo predictor.py, khởi tạo từng nhánh từ checkpoint, tiến trong một khoảng tương lai ngắn. Chấm phôi có tới vùng gắp, có timeout, có nguy cơ rơi và thời gian hoàn thành. Bản dự báo không được sửa plant hoặc đọc tương lai thật do lịch lỗi của evaluator giữ riêng.

Bước 3. Viết supervisor

Tạo supervisor.py để chọn hành vi được phép. Ban đầu chỉ cần tiếp tục hoặc HOLD. Sau khi kiểm chứng mới thêm chính sách ngừng cấp, thay lịch hoặc điều tốc. Hệ thật dùng relay ON/OFF thì không được giả định có điều tốc liên tục như trong mô phỏng.

Bước 4. So sánh có và không có supervisor

Cho hai phương án chạy cùng điều kiện đầu và lịch lỗi. So sánh tỷ lệ gắp đúng, phôi rơi, thời gian dừng và thời gian hoàn thành. Benchmark không giao diện; bỏ --view để tránh phần chờ phục vụ hiển thị. Báo thời gian tính p95/p99 so với deadline, không chỉ xem animation mượt.

Hoàn thành A8: chứng minh nhánh dự báo không thay đổi plant, kết quả tới trước deadline và supervisor có lợi theo tiêu chí đã chọn. Các phương án A/B/C là thiết kế cần bổ sung, chưa phải nút có sẵn trong viewer.

A9 Bảng tự kiểm tra và cách học tiếp

Bài	Chỉ chuyển tiếp khi làm được
A1	Tìm đúng thư mục, hiểu vai trò MuJoCo/Python/WSL/CSV.
A2	Tự chạy hộp 1, có PASS, lưu được CSV.
A3	Đọc được phase, thời gian, PS2 và thời điểm hút/nhả.
A4	Có bảng tín hiệu và tách dữ liệu đo khỏi đáp án đánh giá.
A5	Máy trạng thái chạy được và vào HOLD khi thiếu phản hồi.
A6	Ước lượng được vị trí, đo được sai số, không dùng đáp án để canh gắp.
A7	Có log normal/lỗi, báo sai, độ trễ và giới hạn chẩn đoán.
A8	Có checkpoint, nhánh độc lập, benchmark và so sánh chính sách.

Từ chuyên môn thường gặp

Plant: máy thật hoặc máy ảo đóng vai máy thật. **Ground truth:** đáp án dùng để chấm. **Estimator:** bộ suy ra trạng thái từ dữ liệu đo. **FSM:** máy trạng thái. **Residual:** độ lệch giữa đo và dự báo. **Replay:** phát lại dữ liệu đã ghi. **Checkpoint:** bản chụp đủ trạng thái để tiếp tục chạy. **Supervisor:** bộ chọn chính sách vận hành.

Những việc không nên làm lẫn trong một lượt

Mỗi lần chỉ sửa một hành vi, lưu phiên bản và ghi cách kiểm tra. Không đồng thời đổi hình học, ma sát, điều khiển và cảm biến rồi kết luận nguyên nhân. Không xóa ground truth: hãy giới hạn ai được đọc. Không lấy số liệu mô phỏng chưa hiệu chuẩn để công bố năng suất hoặc độ chính xác của máy thật.

Tra phần lý thuyết khi cần

A4 xem phần 04; A5 xem 05; A6 xem 03 và 06; A7 xem 07, 08 và 11; A8 xem 09, 10 và 11. Phần 12 là lịch triển khai gợi ý, phần 13 nói về đo thực và kết nối PLC, phần 14 cung cấp chủ đề cần học và nguồn tham khảo.

Việc làm trong buổi đầu: chỉ hoàn thành A2 và A3. Khi có log và đọc được dữ liệu, mới bắt đầu thay cách chương trình quan sát hệ thống ở A4. Hướng dẫn hiện giải thích công việc cần làm; việc hoàn thiện các module tương lai vẫn cần viết và kiểm tra mã.

01 Mốc hiện tại và đích cần đạt

Hạng mục	Đã có	Còn thiếu
Mô hình	URDF 5 link, 4 khớp; MJCF hai băng tải và 6 hộp.	Hiệu chuẩn kích thước, khối lượng, ma sát, hành trình.
Chuyển động	Chu trình một phôi: đẩy, vận chuyển, gắp, thả, về gốc.	Nhiều phôi, hàng đợi, phục hồi sau lỗi, chế độ vận hành.
Dữ liệu	PS1/PS2, vị trí 4 khớp, vị trí phôi mô phỏng và lệnh hút.	Tín hiệu hành trình nhị phân, chất lượng dữ liệu, timestamp, heartbeat.
Dự báo	Chưa triển khai lõi dự báo độc lập.	Ước lượng trạng thái, checkpoint, nhánh kịch bản và đánh giá rủi ro.
Đồng bộ	Chưa kết nối PLC hoặc bộ truyền dữ liệu.	Hợp đồng dữ liệu, phát lại, trễ/mất/đảo thứ tự gói, đồng hồ.

Các giá trị hiện tại trong mã nguồn

Bước tích phân vật lý 2 ms; Cell.step() mặc định tiến 10 bước, tương đương 20 ms mô phỏng. CSV hiện ghi xấp xỉ mỗi 100 ms. Băng tải thử ở 0,16 m/s; tốc độ đặt XY/A2 0,12 m/s, A1 0,14 m/s. A1 đặt đích 110 mm; A2 hạ 60 mm trong giới hạn 75 mm. Đây là cấu hình mô phỏng, chưa phải thông số máy thực.

Bằng chứng nghiệm thu hiện có

Đã thử riêng từng hộp 1-6 với một phôi mỗi lần reset. Lượt có giao diện gần nhất: hộp 1 hoàn tất sau 13,58 giây mô phỏng; hộp 2 sau 15,50 giây. Các giá trị này chưa bao gồm đánh giá lặp nhiều lần, năng suất liên tục hoặc so sánh với PLC.

Điểm cần sửa trước tiên: chu trình hiện dùng product_xyz_m để dừng băng tải đầu và tính vị trí canh gắp. Đây là dữ liệu toàn tri của mô phỏng. Nếu dùng nó cho thuật toán phát hiện lỗi thì hệ sẽ biết nhiều hơn hệ vật lý và kết quả không phản ánh khả năng triển khai.

02 Kiến trúc lõi cần xây dựng

Luồng xử lý	Trách nhiệm
1. Plant simulator	MuJoCo đóng vai máy thực ảo; nhận lệnh và tiến động lực học.
2. Sensor model	Sinh PS1/PS2, hành trình A1/A2, home XY, chất lượng tín hiệu; thêm lỗi khi thử nghiệm.
3. Transport	Đóng gói dữ liệu; chèn trễ, jitter, mất, lặp hoặc đảo thứ tự gói.
4. State estimator	Ước lượng vị trí phôi, trạng thái chu trình, độ tin cậy và tuổi dữ liệu.
5. Predictive twin	Khởi tạo từ trạng thái ước lượng; chạy trước các kịch bản, không sửa plant.
6. Supervisor	Chọn chính sách đã cho phép; gửi đề xuất có thời hạn và nhận xác nhận.
7. Sequence controller	Thực thi máy trạng thái và liên động; về sau có thể do PLC đảm nhiệm.
8. Evaluator / logger	So sánh dự báo với ground truth, ghi sự kiện và báo cáo chỉ số.

Tách hai bản sao mô hình

Một bản là plant chạy theo thời gian thực hoặc theo tick giả lập. Bản còn lại là twin dùng để ước lượng và dự báo. Giai đoạn đầu cả hai có thể chạy trên cùng máy WSL, nhưng không chia sẻ trực tiếp vị trí thật của phôi qua một biến toàn cục. Chỉ bộ đánh giá được đọc ground truth.

Giao diện tối thiểu, chưa cần ROS 2

Bắt đầu bằng các đối tượng Python và hàng đợi có timestamp. Khi logic ổn định mới đưa luồng này qua socket hoặc ROS 2. Việc chọn middleware không thay thế mô hình đồng bộ hay thuật toán dự báo. Unity có thể nhận trạng thái để hiển thị trong giai đoạn sau; không nên để tốc độ khung hình quyết định thời gian vật lý.

Tệp dự kiến: sensor_model.py, signals.py, sequence.py, estimator.py, transport.py, predictor.py và supervisor.py. Đây là đề xuất tổ chức cho bước tới, các tệp này chưa được triển khai trong phiên bản hiện tại.

03 Mô hình toán học đủ dùng

Hệ lai: trạng thái liên tục và trạng thái rời rạc

```
x[k+1] = f_dt(x[k], u[k], theta) + w[k]
y[k]   = h(x[k], mode[k]) + v[k]
mode[k+1] = g(mode[k], events[k], guards[k])
```

x gồm vị trí/vận tốc XY, A1, A2, vị trí/vận tốc phôi và trạng thái tiếp xúc. u gồm lệnh động cơ, van, vận tốc đặt và lệnh hút. theta gồm ma sát, tải, thời gian đáp ứng và hệ số động lực học. w và v biểu diễn sai số mô hình và phép đo. mode là bước chu trình, ví dụ TRANSFER, PICK, PLACE hoặc HOLD.

Băng tải và thời điểm phôi tới vị trí gắp

```
s_hat(t) = s_sensor + v_hat * (t - t_sensor)
T_arrival = (s_pick - s_hat) / v_hat,  v_hat > 0
sigma_s^2 ~= sigma_s0^2 + dt^2*sigma_v^2 + v_hat^2*sigma_t^2
```

Công thức cuối giả sử các sai số độc lập và vận tốc gần hằng. Khi phôi trượt, cần thêm sai số quá trình. Trong mô hình, PS1 và PS2 cách nhau 0,235 m; ở 0,16 m/s thời gian đi lý tưởng là 1,469 giây. Kích thước và vị trí này chưa được đo ngoài thực tế.

Trục robot và khí nén

Ở mức đầu, dùng đáp ứng trễ và giới hạn vận tốc/gia tốc cho XY; xi lanh dùng hai trạng thái đầu-cuối cùng thời gian chuyển. Với nghiên cứu lực: M(q)qdd + C(q,qd)qd + g(q) = tau + J(q)^T f_contact. Lực xi lanh lý tưởng F = pA - F_ma_sat - F_tai; lực hút gần đúng F_hut = delta_p * A_hieu_dung. Các công thức áp suất chỉ có ý nghĩa dự báo sau khi biết áp suất, tiết diện, tải và rò rỉ.

MuJoCo hiện dùng lực bám tiếp xúc 5 N để vận chuyển phôi. Không được coi trạng thái lệnh hút là phép đo áp suất hoặc bằng chứng gắp thành công. Nguồn nền tảng mô phỏng: [1].

04 Chuẩn hóa tín hiệu trước khi nối PLC

Nhóm	Trường bắt buộc / quy tắc
Định danh	schema_version, run_id, source_id, sequence_number; số thứ tự riêng cho từng nguồn.
Thời gian	t_source, t_receive, clock_domain; ghi đơn vị rõ. Dùng đồng hồ monotonic để đo khoảng thời gian nội bộ.
Tín hiệu đo	PS1, PS2; A1_retracted/extended; A2_up/down; home X/Y; vị trí trục chỉ khi nguồn thực có thể cung cấp.
Chất lượng	good / stale / missing / suspect / unavailable cho từng tín hiệu; không thay missing bằng False.
Lệnh	command_id, target, created_at, valid_until, ack_id, applied_at và lý do từ chối.
Đánh giá riêng	truth_product_xyz, fault_type, fault_start; không đưa vào đầu vào estimator hoặc supervisor.

Ví dụ một sự kiện cảm biến

```
{"schema_version":1, "run_id":"r001", "seq":42,
 "source_id":"plant_sim", "clock_domain":"sim",
 "t_source_s":5.82, "t_receive_s":5.86,
 "signal":"PS2", "value":true, "quality":"good"}
```

Phân biệt tín hiệu có sẵn và tín hiệu giả định

Người dùng xác nhận xi lanh có cảm biến hai đầu và có hai cảm biến quang. Encoder đo vận tốc băng tải, phản hồi vị trí trục liên tục và cảm biến áp suất chân không chưa được xác nhận. Lệnh xung động cơ bước chỉ cho phép tính vị trí danh định, không chứng minh cơ cấu đã tới đó khi xảy ra mất bước.

Lấy mẫu và bắt cạnh

Bắt cạnh PS và ghi thời điểm ngay khi thay đổi, bên cạnh snapshot định kỳ. Không chỉ dựa vào CSV 100 ms: phôi 22 mm đi ở 0,16 m/s che tia lý tưởng khoảng 137,5 ms, nhưng hình học thực có thể tạo xung ngắn hơn. Phải kiểm tra xung ngắn nhất ở tốc độ lớn nhất và cấu hình bộ lọc/debounce phù hợp.

05 Máy trạng thái điều khiển

Tách hàm commissioning dài thành máy trạng thái có thể tiến từng tick. Mỗi trạng thái có điều kiện vào, lệnh tác động, điều kiện hoàn thành, timeout và đường phục hồi. Đây là cơ sở để tạm dừng, phát lại và tự tái cấu hình.

Trạng thái	Hoàn thành / liên động chính
INIT / HOME	Kiểm tra tín hiệu; A1 thu, A2 trên, XY home theo quy trình đã xác nhận.
FEED	Băng tải đầu cấp phôi; kết thúc theo tín hiệu đo được hoặc ước lượng đã kiểm định.
TRANSFER	A1 đẩy; chờ cảm biến ra; thu A1 trước khi bước tiếp cần khoảng trống.
TRACK	PS1 rồi PS2 đúng thứ tự; ước lượng vận tốc và vị trí phôi.
ALIGN / PICK	Phôi trong vùng gắp, trục ở vị trí cho phép; hạ A2, bật hút, xác nhận điều kiện gắp.
LIFT / MOVE	A2 đã lên; di chuyển XY tới hộp được phép; theo dõi nguy cơ rơi.
PLACE / RETURN	Hạ, nhả, nâng; xác nhận kết thúc; cập nhật đếm phôi và về vị trí chờ.
HOLD / FAULT	Ngừng cấp thêm phôi, lưu lý do và trạng thái; phục hồi phải có điều kiện rõ.

Cách xác định timeout

```
T_timeout = quantile_0.99(T_normal) + margin
transition_allowed = guard_ok AND data_fresh AND interlocks_ok
```

Chưa có dữ liệu để xác định quantile: ban đầu dùng giá trị bảo thủ có ghi nhãn giả định, sau đó đo và cập nhật. Không bỏ qua cảm biến hành trình lỗi chỉ vì bộ dự báo cho rằng xi lanh đã tới. Nếu không có bằng chứng độc lập đủ tốt, chuyển HOLD.

Vấn đề cần xử lý ngay ở băng tải đầu

Hiện không có cảm biến riêng xác nhận phôi đã tới mặt đẩy A1 trong danh sách tín hiệu chắc chắn. Trước mắt có thể dùng thử nghiệm cấp một phôi tại vị trí ban đầu đã biết và timer có sai số; ghi rõ đây là giả thiết thí nghiệm. Muốn cấp ngẫu nhiên liên tục phải kiểm tra vùng quan sát của hai PS hoặc dùng camera; không tự thêm một cảm biến ảo rồi xem như máy thực đã có.

06 Ước lượng trạng thái từ cảm biến

Bản đầu dùng mô hình sự kiện và khoảng tin cậy

PS1 lên cạnh xác nhận phôi đi qua mốc s1. PS2 lên cạnh xác nhận mốc s2; nếu cùng một phôi và không đổi tốc độ giữa hai mốc, v_hat = (s2-s1)/(t2-t1). Giữa các cạnh, dự đoán vị trí theo vận tốc đặt cùng hệ số trượt đã hiệu chuẩn. Mỗi lần không có quan sát, mở rộng khoảng sai số.

Nếu chỉ một PS còn tốt và không có encoder/camera, vị trí vẫn có thể được dự đoán ngắn hạn nhưng độ bất định tăng. Phải công bố giới hạn này. Sau một thời gian hoặc khoảng sai số đủ lớn, ngừng gắp thay vì tiếp tục với một con số có vẻ chính xác.

Kalman/EKF chỉ dùng khi phù hợp

```
x = [s, v]^T; A = [[1, dt], [0, 1]]
x_prior = A*x; P_prior = A*P*A^T + Q
K = P_prior*H^T * inv(H*P_prior*H^T + R)
x_post = x_prior + K*(z - H*x_prior)
P_post = (I - K*H)*P_prior
```

z là phép đo liên tục hoặc vị trí mốc được suy từ cạnh cảm biến. PS là tín hiệu nhị phân; không đưa trực tiếp 0/1 vào bộ lọc vị trí như thể đó là mét. Với nhiều phôi cần ghép đúng sự kiện với ID phôi; khi không phân biệt được thì báo mơ hồ.

Ví dụ định lượng độ tin cậy

Ở 0,16 m/s, sai số thời điểm 20 ms tạo riêng sai số quãng đường 3,2 mm. Sai số vận tốc 10% tích lũy trong 0,5 giây tạo thêm 8 mm. Đây là ví dụ tính toán, chưa phải sai số đã đo. Muốn gắp trong cửa sổ ±10 mm cần kiểm soát tổng sai số, không chỉ tăng tần số hiển thị.

Nghiệm thu bộ ước lượng

Cho phôi chạy với các tốc độ giả lập 0,08 / 0,12 / 0,16 m/s và hệ số trượt khác nhau. Estimator chỉ đọc tín hiệu cho phép; evaluator đọc ground truth để tính MAE/RMSE, sai số p95 và tỷ lệ khoảng tin cậy bao được vị trí thật. Chạy lại cùng log phải cho kết quả tương đương.

07 Đồng bộ thời gian và nhận thức mạng

Ba loại thời gian phải giữ riêng

Thời gian mô phỏng cho biết plant đang ở bước vật lý nào. Thời gian monotonic trên máy tính đo thời lượng xử lý. Timestamp từ PLC hoặc máy khác có miền đồng hồ riêng. Không trừ trực tiếp hai đồng hồ chưa đồng bộ để kết luận trễ một chiều.

```
age = t_now - mapped(t_source)
RTT = t_reply_received - t_request_sent
T_total = T_acquire + T_transport + T_queue
        + T_estimate + T_predict + T_apply
```

Ước lượng offset đồng hồ và độ bất định trước khi tính age giữa hai thiết bị. RTT/2 chỉ là xấp xỉ trễ một chiều khi đường truyền gần đối xứng. Phải đo p50/p95/p99 và tải CPU, không chỉ số trung bình.

Bộ giả lập mạng ở lớp ứng dụng

Yếu tố	Mức thử nghiệm đề xuất, không phải kết quả đo
Trễ	0 / 20 / 50 / 100 / 200 ms, phân phối phải ghi rõ.
Mất gói	0 / 1 / 5 / 10%; thử cả mất ngẫu nhiên và mất theo cụm.
Jitter	Ví dụ độ lệch chuẩn 10 hoặc 30 ms; cắt giá trị âm nếu phân phối cho ra.
Tính toàn vẹn	Lặp gói, đảo thứ tự, ngắt kết nối 0,5-2 s; ghi seed và lịch fault.

Bộ nhận kiểm tra seq, bỏ bản lặp, xử lý dữ liệu cũ và không cho gói đến muộn ghi đè trạng thái mới. Giới hạn age phải xuất phát từ cửa sổ vị trí/thời gian cho phép của thao tác gắp.

ROS 2 QoS hỗ trợ chính sách reliability, deadline, lifespan và liveliness [2]; các chính sách này không tự chèn trễ 200 ms hay mất gói 5%. Dùng transport mô phỏng trước, hoặc công cụ netem khi cần gây nhiễu ở lớp mạng [3]. Cấu hình ở lớp gói không tương đương tỷ lệ mất bản tin ứng dụng khi transport có truyền lại.

08 Phát hiện và phân biệt lỗi

Bất thường	Dấu hiệu	Giới hạn suy luận / phản ứng
PS kẹt OFF	Phôi dự kiến tới nhưng không có cạnh trong cửa sổ.	Cũng có thể phôi không tới, trượt hoặc gói bị mất; kiểm tra seq, PS còn lại, camera nếu có.
PS kẹt ON	Che tia kéo dài hơn khoảng cho phép.	Có thể phôi dừng thật trước tia; chỉ kết luận khi có bằng chứng đối chiếu.
Tốc độ đổi / trượt	Thời gian PS1→PS2 lệch dự báo.	Kiểm tra lệnh tốc độ, thời gian và ghép ID; chưa đủ để chẩn đoán cơ khí.
Xi lanh chậm / kẹt	Van đã tác động nhưng không có hành trình cuối sau timeout.	Phân biệt lỗi cảm biến, van và cơ cấu cần quan sát bổ sung; chuyển HOLD.
Mạng gián đoạn	Heartbeat cũ, thiếu seq, RTT tăng.	Giảm mức tin cậy; hết thời hạn dữ liệu thì ngừng áp dụng đề xuất mới.
Twin sai mô hình	Sai số dự báo có thiên lệch kéo dài dù dữ liệu còn mới.	Đánh dấu twin không đáng tin; quay về chính sách nền đã kiểm chứng.
Mất hút / rơi phôi	Quan sát độc lập cho thấy phôi không đi theo đầu hút.	Lệnh hút không phải xác nhận; không có camera/Vacuum_OK thì khả năng phát hiện hạn chế.

Residual và bộ phát hiện ban đầu

```
r = measurement - predicted_measurement
z = r / sqrt(prediction_variance + measurement_variance)
alarm = (abs(z) > threshold) for M consecutive samples
```

Với sự kiện, residual có thể là t_PS2_thuc - t_PS2_du_bao. Chọn threshold và số mẫu M từ dữ liệu normal cùng dữ liệu lỗi; càng đòi nhiều mẫu, độ trễ phát hiện càng tăng. Các giả thuyết lỗi cạnh tranh cần được giữ song song khi chưa đủ khả năng phân biệt.

Bắt đầu bằng luật có thể giải thích; chỉ bổ sung ML khi tập dữ liệu và phép so sánh nền đã đủ. Nhãn lỗi phải nằm trong logger/evaluator, không lọt vào đầu vào bộ phát hiện.

09 Dự báo nhiều kịch bản từ cùng trạng thái

Chu trình mỗi lần dự báo

1. Chốt snapshot tín hiệu hợp lệ và trạng thái ước lượng tại một mốc. 2. Tạo checkpoint. 3. Tạo các nhánh thay đổi tốc độ, độ trễ, tình trạng PS hoặc chính sách. 4. Chạy mỗi nhánh trong horizon H. 5. Đánh giá rơi phôi, gắp hụt, timeout và thời gian hoàn thành. 6. Trả đề xuất có hạn sử dụng trước deadline.

Checkpoint phải chứa đủ trạng thái

MuJoCo cung cấp API lấy/đặt trạng thái [1]. Ngoài qpos/qvel cần xét act, warm-start và trường tích phân phù hợp phiên bản. Lưu thêm controls, goals, mode/timer của controller, trạng thái estimator, các tham số model đã thay đổi, hàng đợi mạng và RNG. Hiện _refresh_constants chỉ phục vụ thay đổi hệ số trong một phiên chạy; chưa phải cơ chế checkpoint đầy đủ cho dự báo.

Các nhánh có ma sát hoặc lực hút khác nhau phải có model riêng hoặc bảo đảm không sửa đồng thời model chung. Kiểm tra một nhánh không làm đổi trạng thái plant và các nhánh khác. Không tăng timestep tùy tiện để lấy tốc độ: cần kiểm tra độ hội tụ và tiếp xúc.

Tính ngân sách trước khi hứa số kịch bản

```
N <= floor((B - T_setup) / T_rollout)   [chay tuan tu]
RTF = simulated_duration / wall_duration
```

Ví dụ minh họa: B = 50 ms, T_setup = 10 ms, mỗi nhánh mất 4 ms thì tối đa 10 nhánh trong chu kỳ đó, chưa có dự phòng. Nếu horizon H = 1 s thì RTF của một nhánh là 250. Các số này chỉ minh họa công thức; chưa benchmark máy hiện tại và không chứng minh có thể chạy hàng nghìn kịch bản/giây.

Bộ kịch bản khởi đầu

Dùng 5-10 nhánh: bình thường; tốc độ danh định ±10%; PS1 lỗi; PS2 lỗi; dữ liệu cũ 50/100/200 ms; chọn HOLD hoặc tốc độ thấp. Tăng số nhánh sau khi đo p95/p99 thời gian tính. Một vài mẫu tham số không chứng minh an toàn cho mọi nhiễu liên tục.

10 Tự tái cấu hình phần mềm

Tái cấu hình trong đề tài này là thay chính sách vận hành trong phạm vi cơ cấu sẵn có. Supervisor chọn một phương án đã kiểm chứng, còn sequence controller thực thi các liên động và giới hạn địa phương.

Điều kiện	Chính sách đề xuất	Điều kiện sử dụng
Băng tải nhanh hơn	Tính lại thời điểm dừng/gắp, giảm tốc nếu driver cho phép.	Cập nhật ước lượng vận tốc; dữ liệu đủ mới.
PS1 đáng ngờ, PS2 tốt	Dùng PS2 làm mốc; giảm cấp phôi, tăng biên thời gian.	Chứng minh vùng gắp đủ tin cậy; nếu không thì HOLD.
PS2 đáng ngờ	Ước lượng ngắn hạn từ PS1 và vận tốc; hoặc dừng gắp.	Chỉ tiếp tục khi sai số vẫn nằm trong cửa sổ gắp được kiểm định.
Mạng xấu / twin chậm	Ngừng nhận tối ưu mới; giữ chính sách nền hoặc HOLD.	Controller cục bộ còn dữ liệu và có hành vi mất kết nối rõ ràng.
Twin sai / nhiều lỗi	HOLD, lưu log, yêu cầu hiệu chuẩn hoặc xử lý.	Không tự động khôi phục khi nguyên nhân chưa được xác nhận.

Không giả định khả năng điều tốc ngoài thực tế

Sơ đồ hiện có K1/K2 điều khiển băng tải theo bật/tắt. Nếu không có driver điều tốc được xác nhận, supervisor chỉ được dùng lệnh ON/OFF và lịch cấp phôi; khả năng thay vận tốc liên tục trong MuJoCo là biến thí nghiệm. Khi ghép phần cứng phải khai báo capabilities cho từng thiết bị.

Hàm chọn chính sách và giao thức xác nhận

```
J(policy) = a*T_cycle + b*P_miss + c*uncertainty
choose minimum J among policies satisfying constraints
proposal = {command_id, policy_id, valid_until, reason}
```

Ràng buộc có ưu tiên cao hơn chi phí. Nếu dự báo hết hạn trước khi nhận ACK, hủy đề xuất; không gửi lặp một hành động cơ cấu chỉ vì mất ACK. Có trạng thái requested / accepted / applied / rejected để biết lệnh nào thực sự được thi hành.

11 Đo đúng trước khi công bố kết quả

Chỉ số	Cách đo / mục tiêu ban đầu đề xuất
Ước lượng vị trí	MAE, RMSE, p95 theo tốc độ và tuổi dữ liệu; mục tiêu thử ban đầu p95 ≤10 mm tại vùng gấp nếu dung sai cơ khí cho phép.
Phát hiện lỗi	Precision, recall, F1 theo từng loại; mục tiêu nội bộ ≥0,95 trên lỗi đã định nghĩa, báo cả số mẫu và khoảng tin cậy.
Thời gian phát hiện	t_alarm - t_fault và t_alarm - t_first_observable; p50/p95/p99. Không áp chung 100 ms cho mọi lỗi.
Báo động giả	Số alarm/giờ vận hành bình thường và alarm/1000 chu kỳ; cần log normal đủ dài.
Dự báo	Sai số thời điểm tới, sai số T_cycle, recall của sự kiện gắp hụt; p95 thời gian tính so với deadline đã chọn.
Khả năng phục hồi	Tỷ lệ chu kỳ hoàn thành, phôi hỏng/rơi, thời gian HOLD và thời gian phục hồi; so với cùng baseline.
Hiệu năng	RTF, số nhánh xong trước deadline, tải CPU/RAM; ghi CPU, MuJoCo, timestep, horizon và viewer on/off.

Thiết kế thí nghiệm và mẫu số

So sánh A: điều khiển tuần tự; B: có ước lượng/phát hiện lỗi; C: có nhận thức mạng và supervisor. Dùng cùng lệnh cấp phôi, lịch fault và seed cho các phương án. Tách dữ liệu hiệu chuẩn, chọn ngưỡng và kiểm thử theo cả phiên chạy, không chia ngẫu nhiên các dòng thời gian liền nhau.

```
Precision = TP/(TP+FP); Recall = TP/(TP+FN)
F1 = 2*Precision*Recall/(Precision+Recall)
Throughput_gain = (Q_new/Q_base - 1)*100%
```

Giảm cycle 20% tương đương tăng throughput 25% khi điều kiện cấp phôi và tải giống nhau. Không nhầm hai phần trăm này. Một chu kỳ thành công không chứng minh tỷ lệ thành công 99%. Xấp xỉ rule-of-three: nếu 300 thử độc lập không có lỗi thì cận trên tỷ lệ lỗi khoảng 1% ở mức 95%; phụ thuộc giữa các lượt làm suy yếu kết luận. Định nghĩa các metric tham khảo [4].

12 Hướng dẫn bắt đầu trong 7 buổi

Buổi	Làm gì	Sản phẩm nghiệm thu
1	Chạy lại mô hình, đọc run_cell.py, đánh dấu mọi nơi đọc product_xyz_m.	Danh sách đầu vào đo được và dữ liệu chỉ cho evaluator.
2	Chốt signals và mô hình cảm biến A1/A2, home; tạo timestamp, seq và quality.	Log normal có snapshot + cạnh; không gán missing thành False.
3	Tách commissioning thành sequence tick; thêm timeout/HOLD.	Reset, chạy, dừng, tiếp tục theo điều kiện; phôi vẫn đúng hộp.
4	Estimator PS1/PS2 + vận tốc danh định; bỏ truy cập vị trí thật khỏi logic gắp.	Báo cáo sai số so với evaluator; mô tả giới hạn quan sát băng tải đầu.
5	Ghi/replay và hàng đợi chèn trễ/mất gói có seed.	Cùng log cho kết quả lặp được; gói cũ không ghi đè gói mới.
6	Luật PS timeout/stuck và mất kết nối.	Ma trận confusion, độ trễ phát hiện, ví dụ trường hợp không phân biệt được.
7	Checkpoint + 5 nhánh dự báo; chọn HOLD hoặc chế độ nền.	Benchmark headless và chứng minh nhánh không tác động plant.

Mỗi buổi là một đơn vị công việc, không cam kết hoàn thành trong một ngày. Người mới có thể cần nhiều buổi cho từng mục. Chưa cần máy thực cho các bài tập này; kết luận về máy thực chỉ có sau hiệu chuẩn và kiểm thử tương ứng.

Lệnh chạy trong WSL từ thư mục gốc kho

```
cd /mnt/c/Users/Admin/Documents/ChatGPT/plc
python3 mujoco/build_cell.py
~/dt-cpps-venv/bin/python mujoco/run_cell.py \
  --commission --bin 1 --log /tmp/cell_bin1.csv
```

Thêm --view nếu cần xem; cửa sổ do Python điều khiển nên nút Run trong viewer không khởi chạy lại chu trình. Đóng cửa sổ và chạy lại lệnh. Benchmark tốc độ phải chạy headless, bỏ --view vì chế độ xem hiện có sleep để tiến gần thời gian thực.

13 Hiệu chuẩn và nối hệ thật ở giai đoạn sau

Đo những gì trước

Đo hệ tọa độ, tâm hai tia PS, cao độ mặt băng và hộp, hành trình A1/A2, phạm vi XY, kích thước/khối lượng phôi. Đo vận tốc băng theo lệnh, thời gian xi lanh ra/vào, thời gian hút/nhả và độ lặp lại. Với cảm biến hành trình nhị phân, không tự suy ra chính xác toàn quỹ đạo xi lanh.

Nhận dạng tham số

```
theta_star = argmin sum_i rho((y_i - yhat_i(theta))^2)
subject to physically plausible bounds on theta
```

Chỉ ước lượng các tham số mà dữ liệu có thể nhận dạng. Ví dụ thời gian PS1→PS2 có thể cho vận tốc trung bình, nhưng không tách riêng hệ số ma sát, tải động cơ và mô men nếu không có quan sát bổ sung. Có thể dùng least_squares có ràng buộc và loss bền vững của SciPy [5]. Kiểm định trên log chưa dùng để khớp tham số.

Tích hợp PLC S7-1200 CPU 1214C DC/DC/DC

Trước khi chọn giao thức, ghi đúng mã CPU, firmware, phiên bản TIA và khả năng giao tiếp đã bật trong dự án. Không mặc định OPC UA có sẵn. Nhóm PLC cần thống nhất DB/tags, kiểu dữ liệu, chu kỳ cập nhật, timestamp, lệnh/ACK và hành vi khi mất kết nối. Trước tiên đọc dữ liệu ở chế độ shadow; sau đó mới thử đề xuất có xác nhận.

Thứ tự mức trưởng thành

M0: mô phỏng một phôi có chu trình. **M1:** lõi sensor-driven, FSM, replay và lỗi giả lập. **M2:** đồng bộ dữ liệu máy thật, có sai số được đo. **M3:** dự báo chạy song song ở shadow mode. **M4:** supervisor được phép thay một tập chính sách có giới hạn và rollback. Hiện hệ đang ở M0; tài liệu này hướng dẫn đi lên M1 trước.

ML và học tăng cường ở đâu?

Sau khi có M1 và bộ đánh giá ổn định, có thể huấn luyện mô hình sai số hoặc chính sách trong mô phỏng. Đầu vào agent phải giống dữ liệu có thể đo ngoài thực tế; dùng domain randomization và kiểm thử ngoài phân phối. Đưa ML/RL vào sớm khi sensor model chưa đúng dễ tạo chính sách khai thác sai lệch mô phỏng.

14 Lý thuyết cần học và tài liệu tham khảo

Ưu tiên	Chủ đề	Dấu hiệu đã hiểu
Học ngay	Hệ tọa độ, đơn vị, động học tịnh tiến và hệ lai/FSM.	Vẽ đúng luồng phôi và giải thích từng guard/timeout.
Học ngay	Lấy mẫu, bắt cạnh, timestamp, sai số và khả năng quan sát.	Giải thích vì sao PS kẹt OFF và mất phôi có thể giống nhau.
Tiếp theo	Ước lượng trạng thái, Kalman cơ bản, nhận dạng và residual.	Tính sai số vị trí theo sai số vận tốc và thời gian.
Tiếp theo	Hệ phân tán, RTT, tuổi dữ liệu, seq, deadline và QoS.	Không dùng timestamp khác miền đồng hồ để đo trễ trực tiếp.
Sau nền tảng	Dự báo theo horizon, điều khiển giám sát, thống kê kiểm định.	Báo được tốc độ/độ chính xác cùng phạm vi thí nghiệm.

Nguồn chính thức - truy cập ngày 23/10/2026

[1] MuJoCo - Simulation và Python

https://mujoco.readthedocs.io/en/latest/programming/simulation.html

https://mujoco.readthedocs.io/en/latest/python.html

[2] ROS 2 - thiết kế Deadline, Liveliness, Lifespan

https://design.ros2.org/articles/qos_deadline_liveliness_lifespan.html

[3] Linux tc-netem - chèn trễ, mất và đảo thứ tự gói

https://www.man7.org/linux/man-pages/man8/tc-netem.8.html

[4] scikit-learn - Metrics and scoring

https://scikit-learn.org/stable/modules/model_evaluation.html

[5] SciPy - least_squares

https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.least_squares.html

Các nguồn trên hỗ trợ cơ chế mô phỏng, QoS, giả lập mạng và phương pháp đo. Các ngưỡng, lịch triển khai và chính sách trong tài liệu là thiết kế đề xuất cho dự án, không phải kết quả đã được các nguồn chứng minh. Các công thức được trình bày theo dạng cơ bản và giả thiết nêu tại từng mục.